

METODI AVANZATI DI PROGRAMMAZIONE

Report tecnico

Alberi di regressione

Anno accademico 2025/2026

Gruppo di lavoro

Davide Fornelli • Francesco Cavaliere • Nicola Cialdella

Il report descrive la versione client/server del progetto di Metodi avanzati di programmazione, dedicato all'apprendimento e all'utilizzo di alberi di regressione.

Introduzione agli alberi di regressione

Gli alberi di regressione sono modelli di apprendimento supervisionato utilizzati nel machine learning e nel data mining. Prevedono una variabile numerica, detta *target*, a partire dai valori di un insieme di attributi descrittivi.

La costruzione suddivide ricorsivamente gli esempi di addestramento, scegliendo split che riducano la dispersione del target nei sottoinsiemi. Gli attributi discreti distinguono i valori del proprio dominio; quelli continui permettono il confronto con una soglia.

I nodi interni rappresentano le condizioni e i rami i possibili esiti. La costruzione termina quando il sottoinsieme è sufficientemente piccolo oppure lo split non produce più di un ramo. Ogni foglia contiene la media dei target degli esempi associati.

La predizione percorre l'albero dalla radice a una foglia e ne restituisce il valore. Nel progetto l'utente sceglie il percorso rispondendo alle domande presentate dal client.

Introduzione al progetto

La versione client/server comprende un **server**, responsabile dei dati e dell'albero, e un **client da console**, che gestisce l'interazione con l'utente.

Server

Il server acquisisce gli esempi da MySQL, costruisce l'albero e lo salva su disco. Può anche ricaricare un albero già appreso. Rimane in ascolto sulla porta indicata, con valore predefinito 8080, e avvia un thread per ogni client. Ogni sessione mantiene dati e albero separati dalle altre connessioni.

Client da console

Il client riceve indirizzo e porta come argomenti di avvio. Permette di apprendere da una tabella oppure di caricare un albero salvato. Mostra quindi i rami selezionabili e trasmette le scelte al server fino a raggiungere una foglia. Visualizza il risultato e permette di ripetere la predizione sullo stesso albero.

Testing

Le verifiche della versione client/server sono realizzate con JUnit 4 e sono organizzate in due moduli separati, uno per il server e uno per il client. La copertura del codice applicativo viene misurata con JaCoCo. Sono presenti prove delle singole classi e prove di integrazione delle interazioni fra componenti.

I test del server utilizzano un driver JDBC simulato e dataset di prova. Questo consente di riprodurre sia i caricamenti validi sia gli errori di accesso ai dati, senza richiedere un database MySQL attivo. Le prove di comunicazione utilizzano socket locali. Di seguito sono indicati gli ambiti verificati e il trattamento delle classi che non possiedono un test dedicato.

Package data

- **Data**: vengono verificati l'acquisizione degli esempi, la lettura degli attributi e del target, l'ordinamento e il controllo degli indici. Sono inclusi casi di tabella assente o vuota, schema non valido e guasti simulati durante l'accesso al database.
- **Attribute**: essendo astratta, viene verificata attraverso le istanze delle sottoclassi. I controlli riguardano nome, indice e rappresentazione testuale.
- **DiscreteAttribute**: vengono verificati il dominio dei valori, il numero di valori distinti e l'iterazione.
- **ContinuousAttribute**: non introduce operazioni ulteriori rispetto ad **Attribute**; il costruttore e i membri ereditati sono esercitati dai test degli attributi e dei dati.
- **TrainingDataException**: sono verificati messaggio e causa, oltre alla sua produzione nei percorsi di errore di **Data**.

Percorso dei sorgenti: `project/mapServer/src/data/`.

Package database

- **DbAccess**: apertura e chiusura della connessione simulata, recupero della connessione e gestione degli errori.
- **TableSchema** e **Column**: lettura dello schema, riconoscimento dei tipi supportati e accesso alle colonne.
- **TableData**: acquisizione delle righe, estrazione dei valori distinti e gestione dei risultati vuoti.
- **Example**: inserimento, lettura e confronto dei valori, rappresentazione testuale e comportamento corrente dell'iteratore.
- **DatabaseConnectionException** ed **EmptySetException**: verifica dei costruttori e delle informazioni conservate.

Le verifiche sono raccolte in `DatabaseClassesTest`. L'enum `QUERY_TYPE`, annidato in `TableData`, è dichiarato ma non utilizzato nelle operazioni correnti e non ha una prova dedicata.

Percorso dei sorgenti: `project/mapServer/src/database/`.

Package tree

Le classi astratte `Node` e `SplitNode` sono esercitate mediante `LeafNode`, `DiscreteNode` e `ContinuousNode`. I test controllano le statistiche delle foglie, gli intervalli degli esempi, la formazione dei rami, le soglie e il confronto fra split. Verificano anche le informazioni della classe interna `SplitInfo`.

Per `RegressionTree` vengono controllati l'apprendimento ricorsivo, la struttura risultante, le regole e le domande prodotte. Sono comprese le prove di salvataggio e caricamento, gli archivi non validi e gli errori di scrittura.

Percorso dei sorgenti: `project/mapServer/src/tree/`.

Package Server

`ServerBootstrapTest` verifica il punto di ingresso `Server` e `MultiServer`: scelta della porta, parametri errati, porta occupata e isolamento di connessioni con handshake non valido.

`ServerOneClientTest` verifica le sessioni attraverso socket locali. Le prove includono acquisizione dei dati, apprendimento, caricamento degli archivi, predizione su più livelli, ripetizione dei nomi non validi e recupero dopo gli errori. Sono controllate anche la separazione dello stato fra client, la chiusura delle risorse e l'eccezione `UnknownValueException`.

Percorso dei sorgenti: `project/mapServer/src/Server/`.

Package map7Client

`MainTestIntegrationTest` verifica il client da console con un server di prova. Sono esercitati i menu di apprendimento e caricamento, le domande successive, il risultato finale e la ripetizione della predizione. Sono inoltre verificati gli argomenti di avvio, gli errori di rete, le risposte non valide e la fine dell'input da tastiera.

Percorso dei sorgenti: `project/mapClient/src/map7Client/`.

Package utility

La classe `Keyboard` dispone di test per la lettura di stringhe, parole, caratteri, valori booleani e numeri. Le prove comprendono valori minimi e massimi, input non validi e gestione degli errori di lettura.

Percorso dei sorgenti: `project/mapClient/src/utility/`.

Risultati dei test

La verifica dei due moduli client/server ha prodotto i seguenti risultati.

Modulo	Test	Fallimenti	Errori	Saltati
Server	66	0	0	0
Client	30	0	0	0
Totale	96	0	0	0

La copertura JaCoCo è calcolata sul codice applicativo di ciascun modulo, non sulle classi di test o sul driver simulato.

Modulo	Copertura delle righe	Copertura dei rami
Server	98,80%	97,88%
Client	97,78%	100,00%

Entrambi i moduli superano le soglie di copertura del 90% per righe e rami. Non risultano test disabilitati o saltati nelle due suite eseguite.

Alcune prove descrivono intenzionalmente il comportamento dell'implementazione corrente, compresi casi limite non corretti. Per esempio, viene verificato che `Example.iterator()` restituisca attualmente `null` e che un intervallo continuo costante possa provocare una `NullPointerException`. Il superamento di questi test non equivale alla correzione di tali comportamenti.

Le verifiche JDBC sono svolte in ambiente simulato: i risultati non attestano l'installazione o la connessione a un server MySQL reale. La copertura indica il codice attraversato dai test e non costituisce, da sola, una garanzia di assenza di difetti per qualsiasi input.